31.5 环境变量

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~215k tokens，输出 ~5k tokens（本节）

配置文件中难免包含敏感信息——API 密钥、数据库密码、第三方服务的令牌。将这些值直接写在 JSON5 文件中既不安全（可能被误提交到版本控制），又不灵活（不同环境需要不同的值）。OpenClaw 通过环境变量替换机制解决这个问题：在配置文件中用 `${VAR_NAME}` 占位，加载时自动替换为对应的环境变量值。本节分析这套机制的实现细节。

31.5.1 环境变量替换（`src/config/env-substitution.ts`）

基本语法

在任何配置字符串值中，可以使用 `${VAR_NAME}` 引用环境变量：

```
{
  models: {
    providers: {
      "vercel-gateway": {
        apiKey: "${VERCEL_GATEWAY_API_KEY}"
      }
    }
  },
  gateway: {
    auth: {
      token: "${OPENCLAW_AUTH_TOKEN}"
    }
  }
}
```

加载时，`${VERCEL_GATEWAY_API_KEY}` 会被替换为 `process.env.VERCEL_GATEWAY_API_KEY` 的值。

变量名规则

环境变量名必须匹配严格的正则模式——仅允许大写字母、数字和下划线，且必须以字母或下划线开头：

```
// src/config/env-substitution.ts

const ENV_VAR_NAME_PATTERN = /^[A-Z_][A-Z0-9_]*$/;
```

这意味着：

- `${MY_API_KEY}` — 合法
- `${OPENCLAW_TOKEN_123}` — 合法
- `${_INTERNAL}` — 合法
- `${myApiKey}` — **不合法**（小写字母）
- `${123_KEY}` — **不合法**（数字开头）

大写限制是一个有意的设计选择——它防止配置作者误将 JSON 键名当作环境变量（如 `${apiKey}`），同时符合 Unix 环境变量的命名惯例。

转义语法

如果需要在配置值中包含字面量 `${...}`（而非变量替换），使用双美元符号转义：

```
{
  prompt: "Use the $${VARIABLE} syntax for templating"
  // 解析后：Use the ${VARIABLE} syntax for templating
}
```

核心替换逻辑

`substituteString` 函数逐字符扫描字符串，处理三种情况：

```ts
// src/config/env-substitution.ts（简化）

function substituteString(value: string, env: NodeJS.ProcessEnv, configPath: string): s
  // 快速路径：无 $ 符号 → 直接返回
  if (!value.includes("$")) return value;

  const chunks: string[] = [];

  for (let i = 0; i < value.length; i++) {
    const char = value[i];
    if (char !== "$") {
      chunks.push(char);
      continue;
    }

    const next = value[i + 1];
    const afterNext = value[i + 2];

    // 情况 1：$${VAR} → 输出字面量 ${VAR}
    if (next === "$" && afterNext === "{") {
      const end = value.indexOf("}", i + 3);
      if (end !== -1) {
        const name = value.slice(i + 3, end);
        if (ENV_VAR_NAME_PATTERN.test(name)) {
          chunks.push(`\${${name}}`);
          i = end;
          continue;
        }
      }
    }

    // 情况 2：${VAR} → 替换为环境变量值
    if (next === "{") {
      const end = value.indexOf("}", i + 2);
      if (end !== -1) {
        const name = value.slice(i + 2, end);
        if (ENV_VAR_NAME_PATTERN.test(name)) {
          const envValue = env[name];
          if (envValue === undefined || envValue === "") {
            throw new MissingEnvVarError(name, configPath);
          }
          chunks.push(envValue);
          i = end;
```

```
            continue;
          }
        }
      }

      // 情况 3: 普通 $ 符号 → 原样保留
      chunks.push(char);
    }

    return chunks.join("");
  }
```

错误处理：MissingEnvVarError

当引用的环境变量未设置或为空字符串时，系统抛出 `MissingEnvVarError` ，附带变量名和配置路径，方便定位问题：

```
// src/config/env-substitution.ts

export class MissingEnvVarError extends Error {
  constructor(
    public readonly varName: string,
    public readonly configPath: string,
  ) {
    super(`Missing env var "${varName}" referenced at config path: ${configPath}`);
    this.name = "MissingEnvVarError";
  }
}
```

例如：如果 `models.providers.vercel-gateway.apiKey` 引用了 `${VERCEL_GATEWAY_API_KEY}` 但该环境变量未设置，错误消息会是：

```
Missing env var "VERCEL_GATEWAY_API_KEY" referenced at config path: models.providers.ve
```

递归替换

`resolveConfigEnvVars` 作为入口函数，通过 `substituteAny` 递归遍历整个配置对象，对所有字符串值执行替换：

```typescript
// src/config/env-substitution.ts

function substituteAny(value: unknown, env: NodeJS.ProcessEnv, path: string): unknown {
  if (typeof value === "string") {
    return substituteString(value, env, path);
  }
  if (Array.isArray(value)) {
    return value.map((item, index) => substituteAny(item, env, `${path}[${index}]`));
  }
  if (isPlainObject(value)) {
    const result: Record<string, unknown> = {};
    for (const [key, val] of Object.entries(value)) {
      result[key] = substituteAny(val, env, path ? `${path}.${key}` : key);
    }
    return result;
  }
  // 原始类型（number、boolean、null）直接通过
  return value;
}

export function resolveConfigEnvVars(
  obj: unknown,
  env: NodeJS.ProcessEnv = process.env,
): unknown {
  return substituteAny(obj, env, "");
}
```

几个设计细节：

1. **路径追踪**： `path` 参数在递归过程中构建完整的配置路径（如 `models.providers.vercel-gateway.apiKey` ），一旦出错就能精确定位。
2. **仅处理字符串**：数字、布尔值、 `null` 直接通过，不会尝试替换。
3. **非破坏性**：函数返回新对象，不修改输入（纯函数语义）。

31.5.2 环境变量收集（`src/config/env-vars.ts`）

config.env 配置段

OpenClaw 允许用户在配置文件中通过 `env` 段预设环境变量，这些变量会在配置加载早期注入到 `process.env` 中：

```
{
  env: {
    // 结构化的变量声明
    vars: {
      "MY_CUSTOM_API_KEY": "sk-xxx",
      "DATABASE_URL": "postgres://localhost/openclaw"
    },
    // 顶级快捷键（向后兼容）
    "NODE_ENV": "production",
    "DEBUG": "openclaw:*"
  }
}
```

collectConfigEnvVars

`collectConfigEnvVars` 函数从配置中收集所有需要注入的环境变量，合并两个来源：

```
// src/config/env-vars.ts

export function collectConfigEnvVars(cfg?: OpenClawConfig): Record<string, string> {
  const envConfig = cfg?.env;
  if (!envConfig) return {};

  const entries: Record<string, string> = {};

  // 来源 1: env.vars 中的显式声明
  if (envConfig.vars) {
    for (const [key, value] of Object.entries(envConfig.vars)) {
      if (!value) continue;
      entries[key] = value;
    }
  }

  // 来源 2: env 下的顶级快捷键（排除保留键）
  for (const [key, value] of Object.entries(envConfig)) {
    if (key === "shellEnv" || key === "vars") continue;  // 跳过保留键
    if (typeof value !== "string" || !value.trim()) continue;
    entries[key] = value;
  }

  return entries;
}
```

两个保留键的处理：

- **`vars`**：这是结构化变量声明的容器，已在第一步处理过，不应作为环境变量名。
- **`shellEnv`**：这是一个布尔值配置项，控制是否将 shell 环境变量传递给子进程，不应被当作环境变量。

在加载流水线中的位置

回顾 23.1 节介绍的 12 步配置加载流水线，环境变量相关的处理出现在第 3、4 步：

```
步骤 3: config.env 注入 → collectConfigEnvVars → 写入 process.env
步骤 4: ${VAR} 替换 → resolveConfigEnvVars → 替换配置中的变量引用
```

这个顺序至关重要——先注入 `config.env` 中声明的变量到 `process.env`，然后再执行 `${VAR}` 替换。这意味着用户可以在 `env.vars` 中定义一个变量，然后在配置的其他位置引用它：

```
{
  env: {
    vars: {
      "MY_API_BASE": "https://api.example.com"
    }
  },
  models: {
    providers: {
      "custom": {
        baseUrl: "${MY_API_BASE}/v1"  // 引用上面声明的变量
      }
    }
  }
}
```

> **衍生解释**：这种"先声明后引用"的设计在配置系统中很常见。Kubernetes 的 ConfigMap、Docker Compose 的 `.env` 文件、GitHub Actions 的 `env` 段都采用类似模式。其核心优势是"单一事实来源"——敏感值只在一个地方定义，其他位置通过引用使用，避免复制粘贴带来的不一致。

本节小结

1. **`${VAR_NAME}` 语法**允许在配置字符串中引用环境变量，变量名必须全大写且匹配 `[A-Z_][A-Z0-9_]*` 模式。
2. **`$${VAR}` 转义**输出字面量 `${VAR}`，用于配置中需要包含模板语法的场景。
3. **MissingEnvVarError** 在变量未设置或为空时抛出，附带变量名和精确的配置路径，便于调试。
4. **递归替换**遍历整个配置对象树，仅对字符串值执行替换，数字和布尔值直接通过。
5. **`config.env` 段**支持两种环境变量声明方式：`env.vars`（结构化）和顶级快捷键（向后兼容）。
6. **加载顺序保证**：先注入 `config.env` 到 `process.env`，再执行 `${VAR}` 替换——允许配置内部自引用。